QoderWork Skills 开发实践：从传统数科到 AI 数科的转型探索–我的Skills进阶之旅

原创 直播技术团队 大淘宝技术 2026年6月26日 01:38 浙江 10人



本文以作者从传统数科向 AI 数科转型的实践为背景，系统阐述了 QoderWork Skills 的开发方法论与工程体系。文章指出 Skill 本质是将领域知识、标准流程及避坑指南封装为 AI Agent 可执行的“数字助手”，并提出了由编排层（SKILL.md）、参数层（config.yaml）、实现层（scripts/）和知识层（references/）构成的四层分离架构，强调通过结构化指令而非单纯代码注入来提升分析效率与输出稳定性。通过对比 Follow Builders 和 Frontend Slides 等案例，以及用户洞察报告、AB 实验分析等自研 Skill 的实战经验，文章总结了 Description 定义、流程编排、配置模板化及渐进式披露等关键开发技巧，旨在通过工程化手段实现团队知识沉淀与标准化，解放人力以聚焦高价值业务决策。



> Skills 可以适用于QoderWork / Qoder CLI / Claude Code / Codex 等主流 AI Agent平台，这里写QoderWork Skills，主要是因为我目前主要基于QoderWork使用、测试我的Skills

你是否想过，将自己的工作流和方法论封装成聪明的“数字助手”，让它自动帮你搞定重复任务?

其实，Skill 本质上是一份清晰、可执行的指令文档，用于明确告诉模型在什么条件下、按什么步骤产出结果 。

如果你尚未接触过 Skills，建议先别急着动手，而是去体验、使用一些现有的 Skills。

从已有的使用中，你能更敏锐地发掘出日常工作生活中确切的需求点和问题点。

参考结构优秀、触发精准的案例，能帮你把 Skill 写成好用的“执行协议”，而非给人看的普通文档。

这样，我们才能更切身地体会到，怎样更好地利用 Skill 为生活、工作和团队赋能 ：）

下面简单列举一些我体验过的 Skills，希望能给你带来灵感！

日常 Skill

- **browser-use**：完整浏览器操控：页面导航、表单填写、数据抓取
- **follow-builders**：一个 AI 驱动的信息聚合工具，追踪那些真正在做产品、有独立见解的人，并将他们的最新动态整理成易于消化的摘要推送给你。
- **frontend-slides**：HTML 演示文稿生成技能，通过“先看再选”的视觉预览引导用户发现风格，最终让 AI 在严格的工程边界内生成零依赖的单文件网页幻灯片（注意产出不要开部署功能，保护数据安全）

工作 Skill

- 数据问答分析技能：该技能主要支持快速问数、数据异动、归因分析、趋势分析等功能。
- 找odps表和odps问数技能：该技能主要支持查找特定的odps报表，并支持基于odps表的直接问数
- 找fbi报表技能
- 报表解读技能：用户提供fbi的报表id，对指定的报表进行内容解读、指标趋势分析、数据异动分析
- 指标巡检技能：用户问题指定fbi的报表id和指标名称，对指定的指标根据用户问题进行异常巡检诊断，生成最终的指标巡检报告
- (超快版)找odps表和FBI数据集：支持找odps表，找fbi数据集，fdp数据准备表和上传表，支持预览数据

元 Skill

在开始编写自己的Skill前可以先安装“元Skill”，节省时间和精力

- **find-skills**：Skill 检索引擎，有需求先搜索，避免重复造轮子，直接复用社区/平台里经过验证的优秀方案
- **skill-creator**：Skill 创作想到，简单描述想法，通过几轮对话就能设计出一个结构完整、符合最佳实践的Skill框架
- **Tool Advisor**：分析当前任务上下文后推荐最优的工具和 Skill 组合

欢迎大家在评论区推荐体验好的Skills~

02 背景：我们为什么要做 Skills?

数科团队的痛点

数科日常工作中有大量**重复但非标准化**的分析任务：每次做用户洞察要重新写分析代码，每次 AB 实验要手动跑一遍检验流程(如果没有接通天塔看板)，每次给产运出报告要重新排版。这些工作耗时但附加值有限。

从 Idealab 到 QoderWork 的演进

我之前在 Idealab AI Studio 上构建了一个对话型的**用户理解助手**，通过 Prompt Engineering + 知识库 RAG 实现了"给数据 → 出洞察"的闭环。这次实践让我意识到：**把领域知识结构化地注入 AI，能显著提升分析效率和输出质量**。

QoderWork 的 Skills 机制把这个思路产品化了——你可以把团队的分析方法论、场景化问题解决思路、报告模板、校验规则写成 Skill，让 AI Agent 按照专业标准执行分析任务。

Skills 的价值定位

维度	人力（没有 Skill）	有 Skill
分析流程	每次从零开始，依赖个人经验	标准化流程，团队知识沉淀
输出质量	不稳定，因人而异	统一模板，质量可控
效率	一次分析 2-4 小时(or 1-2个人日)	5-15 分钟完成
知识传承	在个人脑子里	文档化、可复用

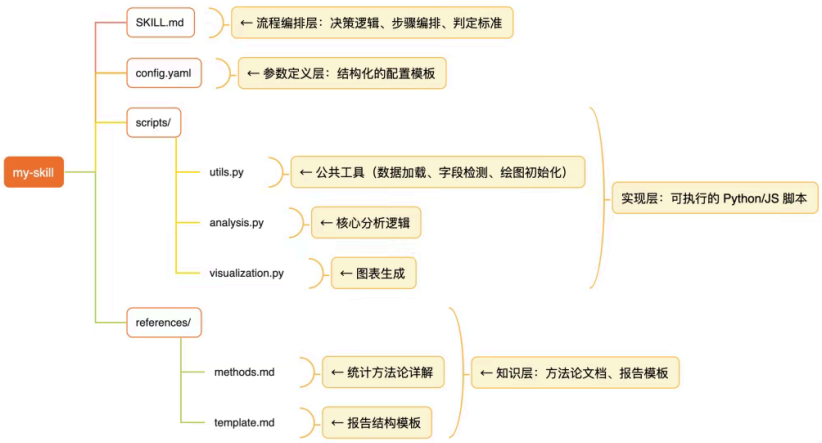
03 Skills 工程体系：不止是一个Skill.md

完整的Skill 结构目录

一个专业的 Skill 不只是一个 SKILL.md 文件，而是一套完整的工程结构：

> 当然很多时候，一个“真正好用”的Skill可能只需要一个md文件就够了，也有很多skills有不同的文件组织和呈现形式，但其核心的设计思路依然是如下所示的工程结构，下文会对一些不同的

> Skill进行拆解和分析
> 大家平时在使用Skill的过程中，也可以让自己的Agent来帮助自己理解一些觉得好用的Skill是如何构建的。



四层分离的设计理念：

层次	文件	职责	关注点
编排层	SKILL.md	流程步骤、决策逻辑、何时调用什么	控制"做什么"和"怎么判断"
参数层	config.yaml	可配置参数的结构定义	控制"可调的旋钮"
实现层	scripts/	具体的代码实现	控制"怎么做"
知识层	references/	方法论依据、模板	控制"为什么这么做"

Skill.md 的定位：编排者而非执行者

- **核心原则：SKILL.md 只负责流程编排和决策指引，不嵌入大段实现代码。**
- SKILL.md 应该回答的问题是：
  - 这个 Skill 什么时候被触发？（触发条件）
  - 分析流程有哪些步骤？（步骤编排）
  - 每一步调用哪个脚本的哪个函数？（实现委托）
  - 遇到异常情况如何决策？（判定标准）
- 不应该出现的内容是大段的 Python 代码——那些应该放在 scripts/ 里。
- **建议篇幅**：控制在 200 行以内（我的两个 Skill 分别是 170 行和 133 行），过长的 SKILL.md 会稀释重点，Agent 反而可能忽略关键指引。

config.yaml的设计哲学：模板而非表单

最初我把 config.yaml 写成了这样：

❌ 错误做法：把具体业务信息写死

```yaml
experiment:
  name: "推荐算法AB实验"            # 写死了实验名称
metrics:
  - name: "GMV"
    field: "gmv"                   # 写死了指标字段
  - name: "观看时长"
    field: "watch_duration_min"    # 写死了指标字段
```

这样的 YAML 不是配置模板，而是一份填好的表单。下次做"直播红包实验"或"搜索排序实验"时，这份配置完全不适用。

正确的做法——参数结构定义模板：

✅ 正确做法：定义结构，运行时填充

```yaml
experiment:
  name: ""                         # [必填] 运行时由用户提供
  description: ""                  # [必填] 运行时由用户提供

fields:
  user_id: auto                    # [自动检测] 扫描列名匹配
  group: auto                      # [自动检测] 扫描列名匹配
  group_values:
    control: auto                  # [自动检测] 扫描数据值匹配
    treatment: auto                # [自动检测] 扫描数据值匹配


metrics: [ ]                        # [必填] 运行时根据数据动态填充
```

- **设计要点：**
  - `auto` 占位符：表示该字段由 scripts/ 中的自动检测逻辑在运行时填充
  - 空列表 `[ ]`：表示该配置项的结构已定义，但具体值在每次运行时动态决定
  - `[默认值]` 标注：有合理默认值的参数（如 `significance_level: 0.05`）可以直接填入
  - 注释说明每个字段的含义、标注 [必填]/[自动检测]/[默认值]

scripts/ 的价值：复杂逻辑的归宿

- Agent 擅长写简单的代码片段，但对于需要精确控制的复杂逻辑（统计检验方法选择、字段自动检测、图表样式），让 Agent 每次临场发挥是不可靠的。
- scripts/ 的作用是**把这些关键逻辑固定下来，确保每次执行结果一致**。
- 以 AB 实验中的字段自动检测为例：

scripts/utils.py 中的核心逻辑

```python
FIELD_PATTERNS = {
    'user_id': ['user_id', 'userid', 'uid', 'uuid', 'device_id', 'visitor_id'],
    'group': ['experiment_group', 'exp_group', 'group', 'variant', 'arm',
              'treatment_group', 'bucket', 'ab_group', 'split', 'test_group'],
    'date': ['date', 'dt', 'event_date', 'timestamp', 'create_time'],
}

def auto_detect_fields(df):
    """扫描 DataFrame 列名和数据值，自动检测字段映射"""
    # 1) 按列名模式匹配标准字段
    # 2) 检测分组值（control/treatment 标签）
    # 3) 识别候选指标（数值型列）
    # 4) 识别协变量（分类型列）
    return detected_fields
```

- 这个函数覆盖了常见的几十种列名变体，保证无论数据来自哪个系统，都能高概率自动映射成功。如果靠 Agent 每次自己猜，准确率和一致性都无法保证。

references/ 的作用：知识的渐进式披露

- SKILL.md 篇幅有限，不可能把每个统计方法的原理都写进去。references/ 提供了**渐进式披露（Progressive Disclosure）**：
  - **SKILL.md** 只说「连续型指标用 Welch's t-test，非正态时回退到 Mann-Whitney U」
  - **references/statistical_methods.md** 详细解释为什么选 Welch 而非 Student、效应量怎么计算、多重比较校正的原理
- 这样 Agent 在正常执行时读 SKILL.md 就够了；当用户追问"为什么用这个方法"时，Agent 可以引用 references/ 中的详细说明。

04 学习好用的Skills的组织形式和优点

Follow Builders

> 一个 AI 驱动的信息聚合工具，追踪那些真正在做产品、有独立见解的人，并将他们的最新动态整理成易于消化的摘要进行推送，可以设置为QoderWork上的每日定时任务
> 功能链：数据抓取 → 内容组装 → AI重新编排 → 投递

- 实际文件结构

follow-builders Skill 文件构成

```
follow-builders/
├── SKILL.md                    ← 编排层（核心大脑）
├── config/
│   ├── default-sources.json    ← 追踪的信息源清单（25位X博主 + 6个播客 + 2个博客）
│   └── config-schema.json      ← 配置校验规则
├── prompts/                    ← 知识层（指导AI如何处理内容的prompt模板）
│   ├── digest-intro.md         ← 整体摘要格式规则
│   ├── summarize-tweets.md     ← 推文摘要指令
│   ├── summarize-podcast.md    ← 播客摘要指令
│   ├── summarize-blogs.md      ← 博客摘要指令
│   └── translate.md            ← 中英翻译指令
├── scripts/                    ← 实现层（确定性代码逻辑）
│   ├── prepare-digest.js       ← 抓取feed + 组装JSON给LLM
│   ├── deliver.js              ← 投递摘要（Telegram/邮件/终端）
│   ├── generate-feed.js        ← 中心化数据抓取（在GitHub Actions中运行）
│   └── package.json
├── feed-x.json                 ← 数据层（预抓取的推文数据）
├── feed-podcasts.json          ← 数据层（预抓取的播客数据）
├── feed-blogs.json             ← 数据层（预抓取的博客数据）
├── .github/workflows/
│   └── generate-feed.yml       ← 自动化：每天UTC 6点GitHub Actions抓数据并提交
└── examples/
    └── sample-digest.md        ← 输出示例
```

- 对比上述设计理念

标准架构	follow-builders 对应	说明
SKILL.md（编排）	SKILL.md（467行）	一样，这个SKILL.md极其详尽，包含了完整的onboarding流程、内容投递工作流、配置管理逻辑
config.yaml（参数）	SKILL.md头部的frontmatter + config/	没有独立的config.yaml，参数定义被简化为SKILL.md头部的两行frontmatter（name + description），用户的个性化配置在运行时生成到 ~/.follow-builders/config.json
scripts/（实现）	scripts/	完全一样。prepare-digest.js 负责数据采集，deliver.js 负责投递
references/（知识）	prompts/ + config/ + feed-*.json	知识层被拆成了三部分：prompt模板指导AI如何总结、config存放信息源清单、feed JSON存放预抓取的原始数据

- 所以**核心区别**并不是"架构不同"，而是：

a. **没有 config.yaml，配置极简化。** 这个 Skill 的 frontmatter 只有 name 和 description 两个字段，所有复杂的用户偏好（语言、频率、投递方式）都在首次运行时通过对话生成，存到 ~/.follow-builders/config.json。

b. **【知识层】被解构了。** 传统Skill把参考资料统一放在 references/，这个Skill把它拆成了三种不同形态——prompt模板（给AI的指令）、JSON feed（给脚本的数据）、config（信息源列表）。

c. **多了一个“中心化数据服务”。** 这是这个Skill最独特的地方：它不要求用户自己配API key去抓推文和播客。作者在GitHub上设了一个每天自动运行的GitHub Actions流水线（.github/workflows/generate-feed.yml），用自己的 X API key 和 Supadata key 抓取数据，结果提交为仓库里的 feed-*.json 文件。用户端的 prepare-digest.js 只需从 GitHub raw URL 拉取这些JSON即可。

Frontend Slides

> HTML 演示文稿生成技能，通过“先看再选”的视觉预览引导用户发现风格，最终让 AI 在严格的工程边界内生成零依赖的单文件网页幻灯片
> 功能链：模式检测(识别是新建、PPT转换还是增强现有文件) → 内容发现(一次性收集：目的、长度、内容、编辑偏好) → 风格发现（生成3个视觉预览）→ 生成交付 → 分享导出
> ⚠️ 注意：含内部数据的文稿生成后请不要选择Vercel部署，以免数据泄露；可以直接让Agent删除部署脚本

- 实际文件结构



- 对比上述设计理念

标准设计	frontend-slides
SKILL.md（编排）	SKILL.md（322行）
config.yaml（参数）	不存在——被"溶解"了
scripts/（实现）	scripts/（1个Python + 2个Bash）
references/（知识）	4个平级文件：STYLE_PRESETS.md + viewport-base.css + html-template.md + animation-patterns.md

- 分层拆解值得学习的部分

1）第一层：编排层 — SKILL.md

a. **核心设计理念：** 作者把 AI 当作一个会遗忘、会走捷径、会趋于平庸的处理器来编程，所以在关键决策点反复设置冗余校验。

b. 几个值得学习的技巧：

i. “NON-NEGOTIABLE”标注法。第 16 行写“Viewport Fitting (NON-NEGOTIABLE)”，然后在第 39-48 行展开规则，第 49 行再强调“read viewport-base.css and include its full contents”，第 62 行兜底“Never cram, never

scroll”。同一条铁律在文件中出现了 4 次不同表述——这不是啰嗦，**而是对抗 AI 在长上下文中注意力衰减的工程手段**。

ii. **反模式清单。** 第 19–35 行不只说“要做什么”，还显式列出“不要做什么”——overused fonts（Inter、Roboto、Arial）、cliched color schemes（purple gradients on white）、predictable layouts。这是**给 AI 设置负向约束**，因为**大模型的“模式坍缩”倾向会让它总是输出最高频的组合，必须显式阻断**。

iii. 内容密度限制表。 第 53–61 行用一张 6 行的表格，给每种 slide 类型规定了硬性内容上限（比如 content slide 最多 4–6 个 bullet）。

iv. “一次性问完”指令。 第 90 行 "Ask ALL questions in a single AskUserQuestion call"。作者深知多轮交互的成本（用户流失、上下文漂移），把 Phase 1 设计成一次性收集所有信息的“表单”。

v. **Gotchas 前置。** Phase 6 把部署和导出的“坑”直接写进了 SKILL.md，而不是放在脚本注释里。这意味着 **AI 在决定是否调用脚本之前就知道可能出什么问题**，能主动提醒用户。

2）**第二层：参数层 — 被"溶解"了**

a. 原因在于：**配置的本质是“提前固化的决策”**。但这个 skill 的所有决策都是运行时通过对话收集的——用户的 mood、slide 数量、内容类型，在 Phase 1–2 实时确定，不存在“提前配置”的场景。

b. 传统 config 层的功能被“溶解”到了三个地方：用户对话（Phase 1–2 收集的偏好，相当于“运行时配置”）、STYLE_PRESETS.md（12 套预定义风格，相当于“枚举型配置”）、viewport-base.css（硬性约束，相当于“不可配置的常量”）。

c. 对比 follow-builders：它需要 config 是因为用户的追踪偏好（语言、频率、投递方式）是**跨会话持久化**的。而 frontend-slides 每次生成演示文稿都是**独立的一次性任务**，用完即走，天然不需要持久化配置。

3)**第三层：实现层 — scripts/**

a. 核心功能（从零生成 HTML 幻灯片）完全不依赖任何脚本——它只靠 SKILL.md 编排 + 知识层的 4 个文件就能工作。

b. 脚本层是纯粹的“可选服务”。*extract-pptx.py* 只在PPT 转换时调用，*deploy.sh* 只在用户选择部署时调用，*export-pdf.sh* 只在用户选择导出 PDF 时调用

c. 当你的“实现层”是 AI 本身时，传统的【 scripts/ 】角色会缩小。 follow-builders 需要 JS 脚本来做 API 调用和数据处理，因为这些是确定性逻辑。但 frontend-slides 的核心任务（生成 HTML/CSS/JS 代码）恰好是 AI 最擅长的事，不需要外部脚本来辅助。

4)**第四层：知识层 — 4个文件的精妙分解**

a. 没有 references/ 子目录，4 个知识文件直接和 SKILL.md 平级放在根目录——因为 Markdown 的 **file 链接语法**在同级目录最简洁，减少路径出错的可能。

b. 4 个文件按关注点分离，各自回答一个不同维度的问题：

文件	知识维度	回答的问题	加载时机
STYLE_PRESETS.md	视觉身份	“长什么样？”	Phase 2
viewport-base.css	物理约束	“什么不能违反？”	Phase 3
html-template.md	结构骨架	“怎么组装？”	Phase 3
animation-patterns.md	动态表现	“怎么动起来？”	Phase 3

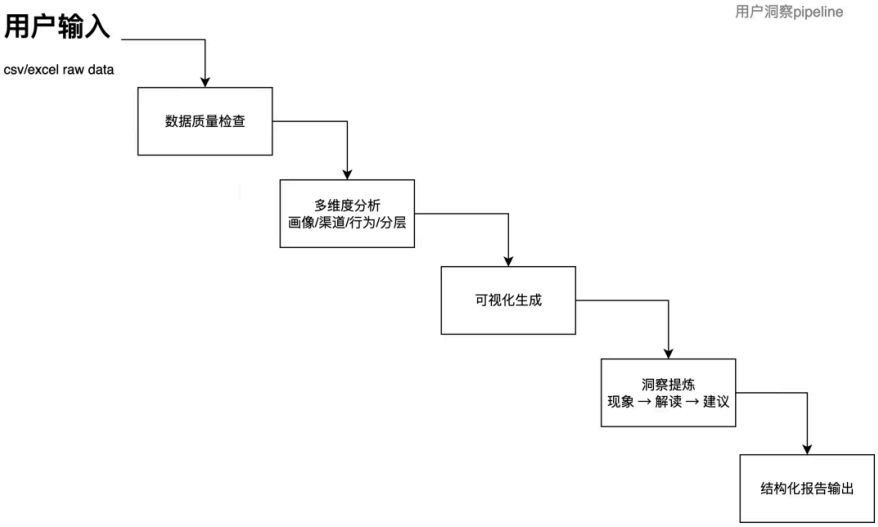
c. 除了动画文件外，其他三个文件存在强依赖关系



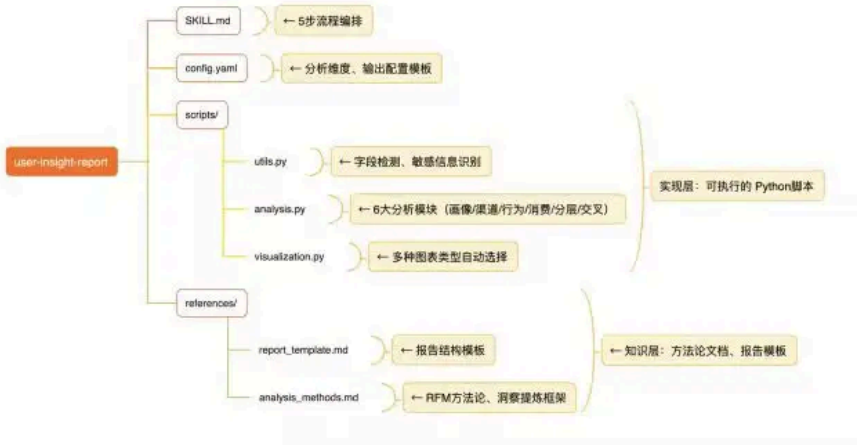
用户洞察报告生成（user-insight-report）

> 因为前期在Idealab 做用户理解助手的实践，先从这个Skill入手，我主要是想体验面对同样的TASK，对话型助手 和 QoderWork Skills会有什么实现上和用户体验上的区别

- **解决的问题**：产运同学经常问“这个场景下的用户是什么样的用户？”，数科需要从原始数据中提炼人群画像、渠道分布、主播/商品偏好等洞察，反哺给业务作为策略信息参考输入。
- **核心流程**：



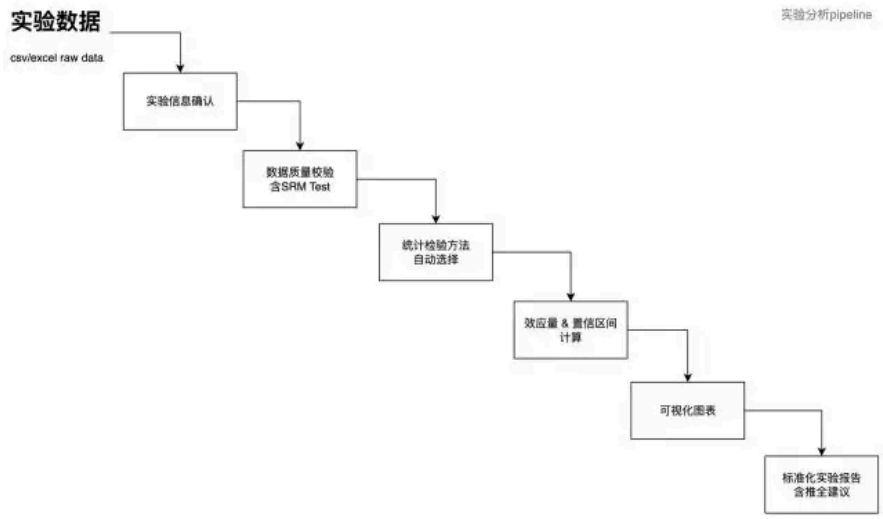
- **工程结构：**



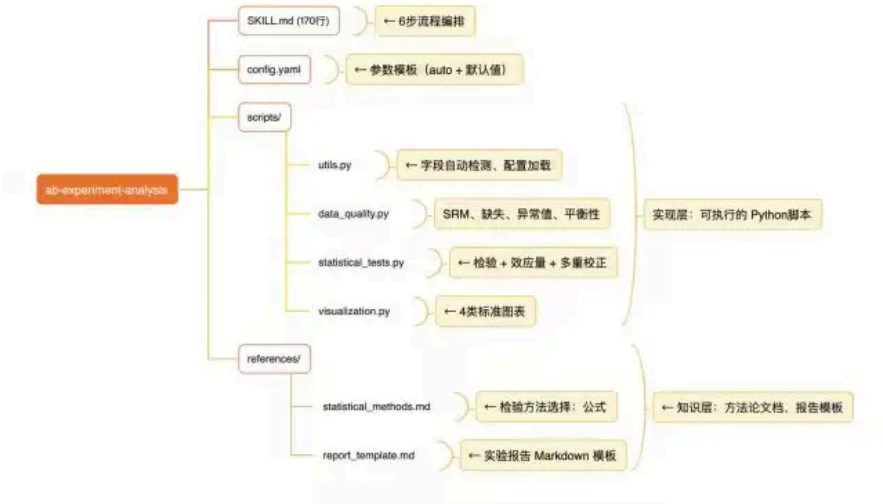
- **设计亮点**：
  - 报告模板同时兼顾产运（业务语言）和数科（统计细节）
  - 每个洞察强制要求"so what"——不只是罗列数字，而是给出业务含义
  - 统一的淘宝橙配色方案，输出图表风格一致
  - **模块化分析**：6 个分析模块按需组合，不是每次都跑全套——数据中有什么字段就做什么分析
  - **敏感信息自动识别**：utils.py 内置正则匹配手机号、身份证等敏感字段，自动脱敏
  - **RFM 自动分层**：当数据中存在 Recency/Frequency/Monetary 相关字段时，自动执行分层
  - **PIA 洞察框架**：每条洞察强制包含 Pattern（现象）→ Interpretation（解读）→ Action（建议）

AB 实验分析（ab-experiment-analysis）

- **解决的问题：**实验分析流程标准化，避免遗漏关键校验步骤（如 SRM 检验），确保结论可信。
- **核心流程：**



- **工程结构：**



- **设计亮点**：
  - **SRM 强制校验**：SRM 检验作为强制步骤，不通过时必须告警，分流比例异常时必须告警，阻断错误结论
  - **检验方法自动选择**：根据指标类型（连续/比率）自动选择检验方法，statistical_tests.py 根据指标类型（连续/比率）和数据特征（正态性、样本量）自动决定用 Welch's t-test、Mann-Whitney U 还是 Z-test

- **结论判定矩阵**：综合 p 值 + 效应量 + 效应方向自动生成推全建议；不只看 p 值，综合考虑效应量和业务意义
- **内置"避坑指南"**：多重比较校正、长尾分布处理、peeking problem 提醒，多重比较校正：分析多个指标时自动 Bonferroni 校正，避免假阳性



Skill 的本质

Skill 不是一个独立的工具或应用，而是**给 AI Agent 的一份"领域专家手册"**。可以把它理解为：

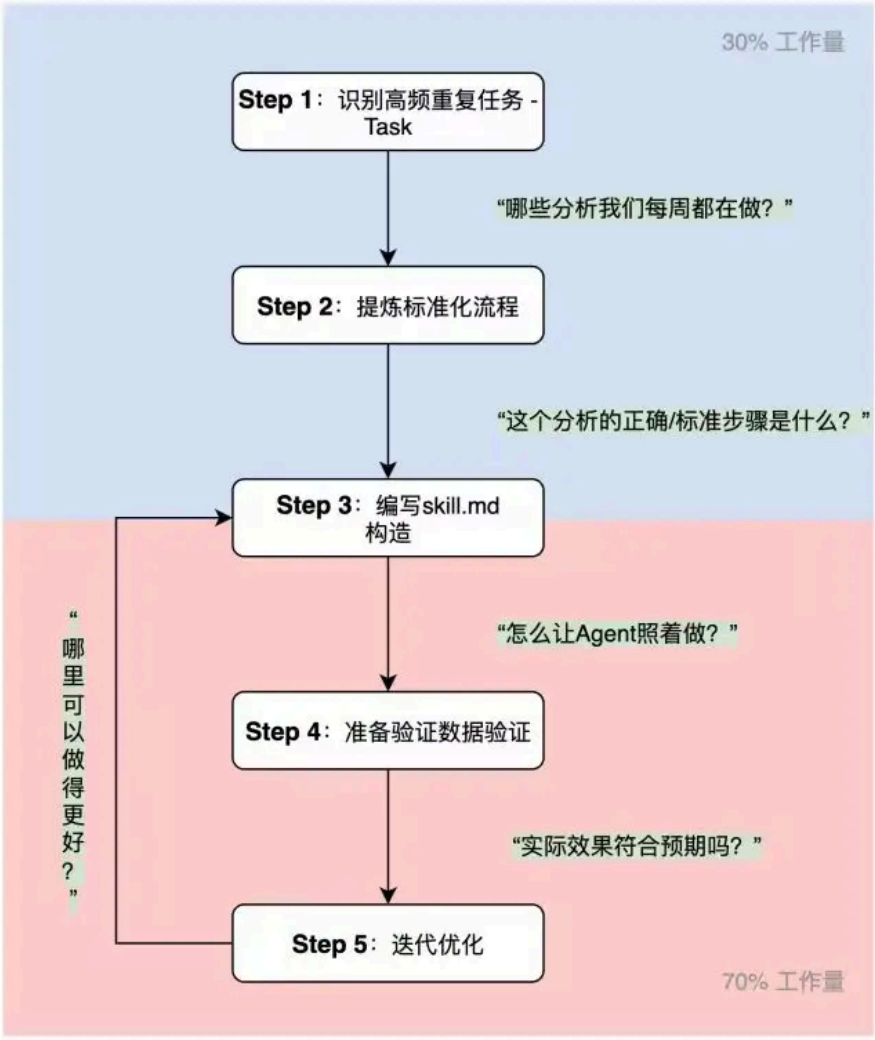
> Skill = 领域知识 + 标准流程 + 输出模板 + 避坑指南

Agent 读取这份手册后，就能按照提供的团队专业标准来执行分析任务。

> Skill 不是“把一个功能介入模型”，而是把一段“可复用、可验证、有边界”的业务流程，封装成一个稳定能力单元

它解决的不是让“模型更聪明”的问题，而是让“系统更可控”的问题

开发流程



编写技巧

- Description 是灵魂

- Description 决定了 Agent 什么时候会调用你的 Skill。
- 写法：明确说 WHAT（做什么）和 WHEN（什么时候用）。
- **必须项：触发条件 ⚠️ （/适用场景）**，写清楚触发条件才能够让Sill有机会被调用

Description

```
# 好的 description
description: 分析用户数据并生成结构化的用户洞察报告。适用于用户画像分析、来源渠道分析、用户偏好洞察报告、用户画像分析时使用此技能。

# 不好的 description
description: 帮助做用户数据分析
```

- 给代码不如给流程

- Agent 本身很擅长写代码，但不擅长把控现实商业世界里的专业数据分析流程。
- SKILL.md 的职责不是"怎么写 Python"，而是"分析应该**包含哪些步骤、关注哪些指标、注意哪些陷阱**"，具体实现交给 scripts/。
- 因此我觉得，SKILL.md的编写是重中之重，至于需不需要给 scripts/，可以具体情况具体分析，但真的都还ok。

- 模板比自由发挥更可控

- 定义**结构化输出**的Schema，Skill一定要有能结构化输出的能力，没有Schema，Skill就会退化成和对话聊天一样的背景板
- 提供明确的报告模板，确保输出格式统一，减少 Agent 的随机发挥带来的不确定性。

- config.yaml 是模板不是表单

- 所有和具体业务相关的值（实验名称、指标列表、字段名）都不应该写死在 config 里，而是用 `auto` 或空值占位，运行时由检测逻辑或用户确认来填充。

- 控制篇幅， 渐进式披露控制信息密度

- SKILL.md 建议控制在 500 行以内（200行以内更佳）。信息太多反而会稀释重点。
- SKILL.md 言简意赅，方法论细节放 references/，代码实现放 scripts/。这样 Agent 不会被信息过载。

07 实践心得

从 Idealab RAG 到 QoderWork Skills 的对比

维度	Idealab RAG 助手	QoderWork Skills
知识注入方式	知识库文档 + Prompt Engineering	SKILL.md + config + scripts + references
执行能力	只能对话	可执行代码、生成文件
输出形式	文本回复	报告文件(MD/Word/PDF) + 图表
迭代成本	Prompt 调优、修改知识库文档	四层架构分离关注点
适用场景	轻量问答	端到端分析任务

关键收获

1. **测试数据很重要，测试驱动开发**：开发 Skill 时一定要用模拟数据跑通全流程，对 Skill 不断进行测试，发现其中的问题，进一步调整优化，这一步的工作可能占据了实际Skill 开发的70%–80%。
2. **产运视角 ≠ 数科视角**：报告模板要考虑非专业背景用户的阅读体验。“p=0.023”对产运没有意义，“实验组 GMV 提升 8%，建议推全”才是他们想看的。
3. **工程化思维**：Skill 开发不是写一个 Markdown 文件的事，而是一套工程体系。config.yaml 如何设计、scripts/ 如何拆分、SKILL.md 如何引用——这些架构决策直接影响 Skill 的通用性和可维护性。
4. **Token 消耗**：对 Skill 进行调用测试验证的过程中，对于Token的消耗量极大，后续需要进一步思考如何在开发过程中节约成本。



结语

- QoderWork Skills 的核心价值在于**把团队的分析方法论 & 典型场景/问题产品化**。对于数科团队来说，这不仅是效率工具，更是**知识管理和标准化的载体**。
- 一个专业的 Skill 不是一个 SKILL.md 文件，而是 SKILL.md（编排）+ config.yaml（参数）+ scripts/（实现）+ references/（知识） 四层协作的工程体系。当然，**很多情况下，一个 SKILL.md 文件就能work**。这样的架构既通过脚本层的管理实现了可控性&稳定性的工程思维，又通过知识层的管理体现了渐进式加载、渐进式披露的上下文管理美学。
- 从 Idealab 的 Prompt + RAG 实践，到 QoderWork Skills 的开发，我最大的体会是：**AI 的天花板取决于你注入的领域知识质量**。技术能力可能已经不再突出化的重要，取而代之的是**思维能力、推理能力、产品能力、Business Sense 和 Ownership**。
- 传统数科向 AI 数科的转型，不是让 AI 替代我们，而是让我们把精力从重复性的“跑数出图出报告”中解放出来，聚焦到更有价值的“定义问题、设计方案、推动落地”上。



本文作者自遂，来自淘天集团–直播技术团队。我们致力通过大数据、机器学习与AI技术打造数据驱动的直播智能决策体系，服务于淘宝直播全链路业务增长，面向用户、主播、商家、商品构建从前瞻洞察到策略落地的端到端解决方案。团队以因果推断与实验平台驱动科学决策，以全栈数据科学能力支撑业务洞察，同时积极建设AI数据科学家能力，通过AI分析与端到端的AI数据能力提升服务效率，助力淘宝直播生态的确定性增长。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 4858

留言

写留言

还没有留言，来写首评